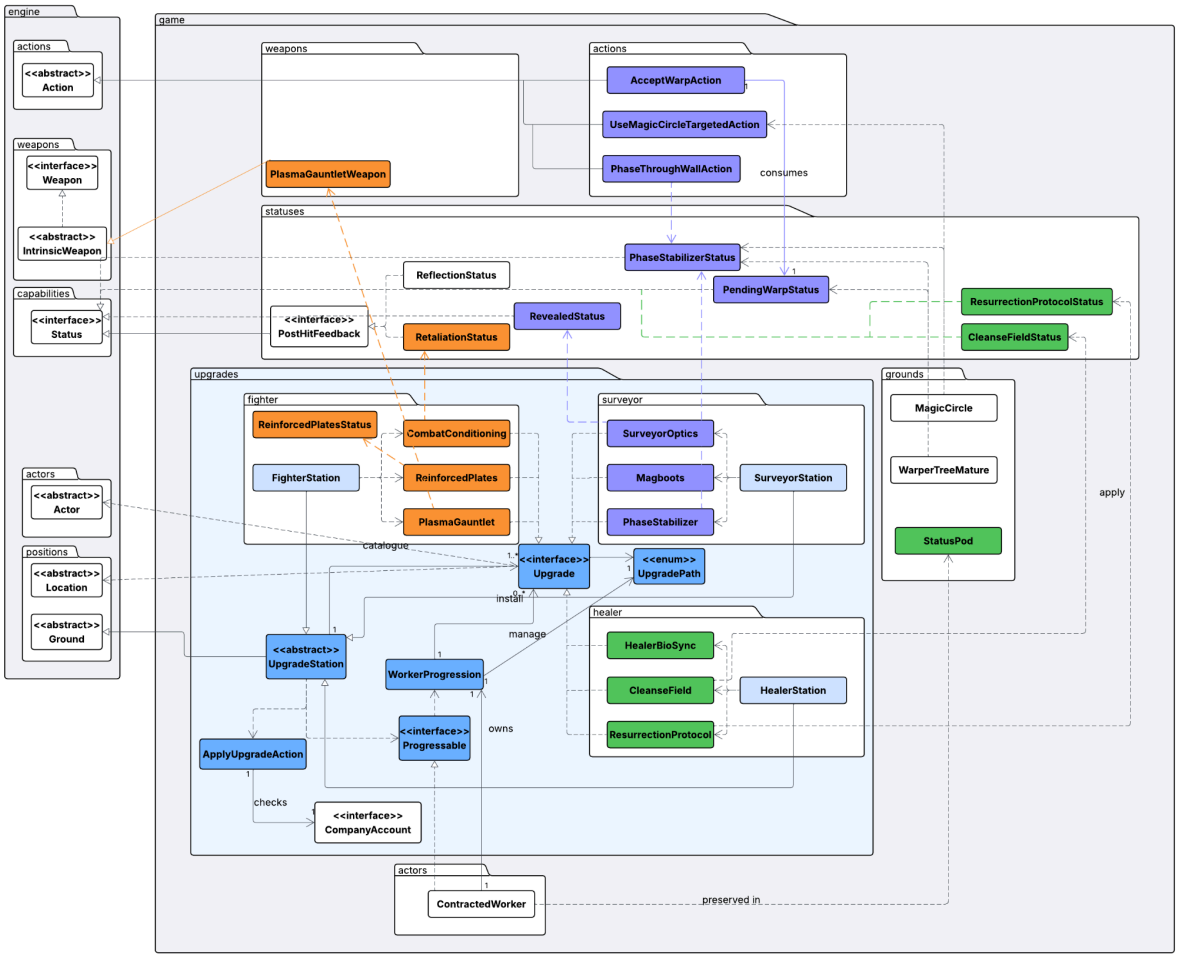
REQ3 — Worker Progression / Upgrade System — Design Rationale



Key Design Decisions

A. How the worker's role-specialisation is modelled

Design 1: Make Fighter, Surveyor, and Healer direct subclasses of ContractedWorker. When a worker buys their first tier-1 upgrade at a Lab Station, the engine replaces the existing worker with a freshly-constructed subclass instance carrying the same name, inventory, and credits.

Pros	Cons
Straightforward is-a modelling — the role is encoded directly in the type.	Specialisation is a runtime, mutable event; Java cannot reclass an existing instance, so the worker has to be destroyed and recreated — losing identity, references held by other actors, and any subtle state in the engine's internal ActorLocationsIterator.
Per-path behaviour can live in the subclass via virtual method dispatch.	Climbing tiers within a path would mean repeating the destroy-and-recreate dance, multiplying the failure surface.
	Three sibling subclasses invite duplication or fragile-base-class problems whenever shared worker logic is touched.
	Adding a fourth path requires editing the worker class hierarchy, violating the Open-Closed Principle.

Design 2: Keep one ContractedWorker class and compose the role via a WorkerProgression field plus the list of installed Upgrade objects. Path identity is an UpgradePath enum value; tier is an int. The State / Strategy pattern (GoF) is the textbook fit for "behaviour that changes with internal state without changing class".

Pros	Cons
Specialisation is mutable at runtime without changing the actor's type. The worker keeps their identity across path-locks and tier-climbs.	The role identity lives in an enum field rather than the type, which is slightly less natural for a reader expecting OOP-101 inheritance modelling.
The worker class stays focused on player-input, menu construction, and damage routing (Single Responsibility Principle); per-feature complexity is housed in the relevant Upgrade concrete.	Two pieces of state (path + tier) are co-managed — they can in principle desynchronise, mitigated by going through WorkerProgression.install exclusively.

Adding a fourth path is a new Upgrade set + a new UpgradeStation concrete; no edits to the worker class (Open-Closed Principle).	
The engine's polymorphic dispatch on statuses, abilities, and capabilities stays simple — no if (worker instanceof Fighter) checks anywhere.	

B. How the Lab Station discovers a standing worker's progression state

Design 1: Use if (actor instanceof ContractedWorker worker) inside UpgradeStation.allowableActions, downcast to the concrete type, and call worker.getProgression() directly.

Pros	Cons
No additional interface to introduce; uses the concrete worker type directly.	The station depends on a concrete class, breaking the Dependency Inversion Principle.
	instanceof is an OOP anti-pattern — bypasses polymorphic dispatch and ties extension to type-tests.
	A future "upgradable" entity (e.g. a recruitable NPC, an automated proxy) would force edits to every place that downcasts.
	Inconsistent with the engine's existing capability-lookup style (getActorAs(Infectable.class)), so the codebase loses a uniform convention.

Design 2: Add a Progressable capability interface (single method: getProgression()). ContractedWorker implements Progressable. UpgradeStation discovers it via the engine's Actor.asCapability(Progressable.class), which returns an Optional<Progressable> — empty if the actor doesn't advertise the capability.

Pros	Cons
The station depends only on the Progressable abstraction, never on ContractedWorker (Dependency Inversion Principle).	A small auxiliary interface that's currently only useful in conjunction with WorkerProgression.

Zero instanceof, zero downcast. The capability lookup is the polymorphic dispatch.	asCapability is final on GameEntity, so Mockito 3.2.4 cannot stub it — REQ3's unit tests use real Actor subclasses via the TestActors fixture (minor extra test cost, documented).
Reuses the same pattern the engine already exposes for Infectable, Location.getActorAs(...), etc. — readers see a familiar idiom.	
New upgradable actor types plug in by implementing Progressable and nothing else — the station stays untouched (Open-Closed Principle).	

C. How the catalogue of upgrades is structured

Design 1: A single Upgrade class parameterised by configuration data — name, path, tier, costs, plus a List<Consumer<Actor>> of effect closures. Stations hold a List<Upgrade> of pre-configured instances; "Combat Conditioning" and "Healer Bio-Sync" are the same class with different data.

Pros	Cons
Adding a new upgrade is a data change; no new class needed.	Effect logic is forced into anonymous functional bodies, which fight the type system — readers cannot navigate to "where does Combat Conditioning live" from the IDE.
The class itself is small.	Cross-component effects (terrain modification, status install, weapon swap, AoE pulse) cannot be expressed cleanly via Consumer<Actor> — they need typed access to the location, the map, and the worker.
	Tests for individual upgrades become test cases for "Upgrade with config X" — diffuse and hard to name.
	Rule 2 of A3 REQ3 demands complex effects per concrete class; data-configured upgrades would arguably present as one concrete with nine configurations, not nine concretes.

Design 2: Upgrade is an interface with applyTo(Actor) and tick(Actor, Location). Each of the nine upgrades (3 paths × 3 tiers) is its own concrete class. The interface is small (identity, costs, two effect hooks); each concrete is typically 30–80 lines.

Pros	Cons
Every concrete satisfies Rule 2 individually with a typed, navigable implementation that the marker can read in seconds.	A per-tier class means we cannot configure new

	variants without code edits.
Effect logic has typed access to Actor, Location, the engine API, and the concrete Status/Ground types it operates on.	Nine .java files for nine upgrades vs one for data-driven variants.
Tests target one concrete each, with method names that read in English (HealerBioSyncTest.healsAdjacentWorkerAlly).	
Each concrete owns one upgrade's worth of logic, satisfying the Single Responsibility Principle. New upgrades plug in by writing one class — the catalogue line on the station is the only consumer-side edit (Open-Closed Principle at the catalogue boundary).	

D. How REQ3 integrates with REQ1's Company Credit pool

Design 1: ApplyUpgradeAction and the stations reference REQ1's QuotaManager singleton directly via QuotaManager.getInstance().deduct(amount).

Pros	Cons
Direct, minimal indirection.	REQ3 cannot compile until REQ1 lands, blocking parallel development.
	A high-level concern (worker upgrades) depends on a low-level concern (a teammate's ledger implementation), inverting the Dependency Inversion Principle.
	Tests of ApplyUpgradeAction must construct or mock the real QuotaManager, importing its dependencies into REQ3's test classpath.
	If REQ1's QuotaManager API changes during development, every consumer in REQ3 must be edited in lockstep.

Design 2: Define a narrow CompanyAccount interface (just canAfford(int) and deduct(int)) inside REQ3's package. ApplyUpgradeAction and UpgradeStation depend on the interface only. A throwaway InMemoryCompanyAccount stub lets REQ3 run end-to-end while REQ1 is still in development; when REQ1 lands, the teammate adds implements CompanyAccount to their quota manager and EclipseNebula.initialise swaps in their instance.

Pros	Cons

REQ3 and REQ1 progress in parallel; neither blocks the other.	A coordination point exists — the REQ1 teammate must remember to add implements CompanyAccount. Mitigated by a manager review at integration time.
Classic Dependency Inversion — both REQ3 (high-level) and REQ1 (low-level) depend on an abstraction owned at the boundary between them.	One throwaway scaffolding class (InMemoryCompanyAccount) lives in production sources until REQ1 lands. Clearly marked in its Javadoc.
Tests of ApplyUpgradeAction mock the interface in one line — no engine state required.	
The two-method contract is small enough to review in seconds and stable enough that REQ1's internal changes never propagate back here.	

E. How the Combat T1 retaliation effect delivers feedback to the attacker

RetaliationStatus needs to apply a BurnStatus to the attacker when the worker is hit. The engine's Actor.hurt(int) does not carry the attacker reference — the attacker is only known inside AttackAction.execute. The pre-existing Swarmling ReflectionStatus solved the same problem via a hand-rolled applyReflection() method inside AttackAction.

Design 1: Add a second hand-rolled applyRetaliation() method to AttackAction mirroring applyReflection(). Both methods inspect the target's statuses for their respective concrete classes (ReflectionStatus, RetaliationStatus) and apply their effects.

Pros	Cons
Minimal change; the new method is structurally identical to the existing one.	AttackAction accumulates one hand-rolled method per post-hit effect — direct coupling to every concrete status class.
	Adding a third post-hit effect (e.g. vampirism) means editing AttackAction again, violating the Open-Closed Principle.
	The same iteration-and-cleanup boilerplate is duplicated across each method (DRY violation).

Design 2: Promote the pattern into a PostHitFeedback interface (sub-type of Status) with a single method onTargetHit(attacker, victim, map). Refactor ReflectionStatus to implement it. AttackAction.applyPostHitFeedback iterates target.statusesOf(PostHitFeedback.class)

polymorphically. RetaliationStatus slots in by implementing the interface — zero edits to AttackAction.

Pros	Cons
AttackAction depends only on the PostHitFeedback abstraction, never on the concrete status classes (Dependency Inversion Principle).	An additional interface to learn before reading AttackAction.
New post-hit effects plug in by writing one class and zero AttackAction edits (Open-Closed Principle).	The refactor touches an existing A2 class (AttackAction) — a small risk of regressing Swarmling tests, mitigated because behaviour is preserved.
The Swarmling reflection mechanism is preserved unchanged but becomes a polymorphic case rather than a hardcoded one.	
Duplication eliminated — the iteration, cleanup, and message-aggregation live once in applyPostHitFeedback (DRY).	

Summary

In each of the five decisions above we chose Design 2 — the option that places a narrow abstraction at the seam between high-level intent and low-level mechanism. The rationale for each choice and the OOP principle it serves:

A. Composition (WorkerProgression) over inheritance. Runtime role-change makes inheritance technically infeasible — destroying and recreating an actor would lose identity, statuses, and engine-internal references. Keeping one ContractedWorker class and composing a role via a held progression object preserves identity and satisfies the Single Responsibility Principle (the worker stays focused on player-controlled-actor concerns) and the Open-Closed Principle (a fourth path costs zero edits to the worker class).

B. Progressable capability lookup via asCapability over instanceof ContractedWorker. The station depends on a narrow capability interface, not the concrete worker class — a textbook application of the Dependency Inversion Principle. New upgradable actor types plug in by implementing Progressable and nothing else (Open-Closed Principle), and the codebase stays consistent with the engine's existing capability-lookup idiom for Infectable, Location.getActorAs(...), etc.

C. Upgrade interface with nine per-tier concrete classes over a single data-configured class. Each upgrade owns one cross-component effect in typed, IDE-navigable code — satisfying Rule 2 individually, Single Responsibility at the concrete level, and Open-Closed Principle at the catalogue boundary (adding an upgrade is one new class plus one new catalogue line).

D. CompanyAccount interface seam over direct reference to REQ1's QuotaManager. REQ3 and REQ1 both depend on an abstraction owned at the boundary between them — Dependency Inversion Principle applied across a requirement boundary. This unblocks parallel development, simplifies testing (mock the interface, no engine state), and survives any internal API changes the REQ1 teammate makes during development.

E. Polymorphic PostHitFeedback abstraction over a hand-rolled applyRetaliation mirror of applyReflection. Generalising the existing Swarmling-reflection pattern means AttackAction accepts any new post-hit effect (retaliation, future vampirism, etc.) with zero edits — Open-Closed Principle via abstraction extraction. The refactor also eliminates a hardcoded coupling from AttackAction to ReflectionStatus, strengthening Dependency Inversion.

The recurring theme is narrow interfaces between high-level intent and low-level mechanism. Three of the five decisions (B, D, E) are direct applications of Dependency Inversion — they each replace a coupling to a concrete class with a dependency on an abstraction the concretes fulfil. The other two (A and C) are Single Responsibility / Open-Closed wins at the class structure level. Together they make REQ3's architecture closed against most kinds of future change (new paths, new upgrades, new post-hit effects, REQ1's quota implementation arriving) while keeping each concrete class compact and individually testable.